

SelfBalancing Robot Firmware:

A Complete Walkthrough of the STM32F3 + Keystudio Kit Implementation

Prepared for Lab 11 – OpenEnded Project

April 27, 2026

Contents

1	Introduction	3
2	Project Overview	3
3	Hardware Setup and Pin Configuration	3
3.1	Microcontroller and Shield	3
3.2	Pin Mapping	4
3.3	Timer and Peripheral Configuration	4
4	System Architecture and Control Loop	5
4.1	Angle Estimation – Complementary Filter	5
4.2	PID Controller	6
4.3	Motor Driver	6
4.4	Encoder Monitoring	6
4.5	Telemetry	6
5	Detailed Code Explanation	6
5.1	File: mpu6050.c & mpu6050.h	7
5.2	File: complementary.c & complementary.h	9
5.3	File: pid.c & pid.h	10
5.4	File: motor.c & motor.h	12
5.5	File: encoder.c & encoder.h	13
5.6	File: main.c	14
6	Extra Features Beyond Lab 11 Requirements	17
6.1	EncoderBased Wheel Speed Measurement	17
6.2	Comprehensive Telemetry Stream	18
6.3	NonBlocking Telemetry Infrastructure	18
6.4	Custom <code>printf</code> Function <code>myPrintf</code>	18
6.5	MPU6050 Calibration Routine	18
6.6	Modular, Reusable Driver Architecture	18
6.7	FixedRate Control Loop in a Timer ISR	19

6.8	Atomic Encoder Data Reads	19
6.9	Architecture Ready for Differential Steering	19
7	Conclusion	19
8	Evaluation Questions: PID Controller for Self-Balancing Robot	21
9	Code Explanation And Overview	25
9.1	Project Architecture Overview	25
9.2	MPU6050 Driver (<code>mpu6050.c/h</code>)	25
9.2.1	Header File (<code>mpu6050.h</code>)	25
9.3	Source Code (<code>mpu6050.c</code>)	26
9.3.1	Internal Helpers	26
9.3.2	Initialisation: <code>mpu6050_init()</code>	26
9.3.3	Reading Sensor Data	27
9.3.4	Calibration	27
9.3.5	Key Takeaways	27
9.4	Complementary Filter (<code>complementary.c/h</code>)	27
9.4.1	Theory	27
9.4.2	Code	28
9.4.3	Public Interface	28
9.5	PID Controller (<code>pid.c/h</code>)	28
9.5.1	What is PID?	28
9.5.2	Header	28
9.5.3	Implementation	29
9.5.4	Gains Used in the Project	29
9.6	Motor Driver (<code>motor.c/h</code>)	30
9.6.1	Header	30
9.6.2	Implementation	30
9.7	Encoder Driver (<code>encoder.c/h</code>)	31
9.7.1	Purpose	31
9.7.2	Header	31
9.7.3	Initialisation	31
9.7.4	Interrupt Callback	32
9.7.5	Reading Data	32
9.8	Main Application (<code>main.c</code>)	32
9.8.1	Configuration and Global Variables	32
9.8.2	System Initialisation: <code>start_robot_modules()</code>	33
9.9	The Control Loop (TIM6 Interrupt)	33
9.9.1	Telemetry	34
9.9.2	Additional Interrupt Service Routines	34
9.10	Summary	34

1 Introduction

The robot is built around an **STM32F3Discovery** board (STM32F303VCT6, clocked at 48MHz) and the **Keystudio SelfBalancing Car Kit V3**. The firmware reads an MPU6050 inertial measurement unit, fuses the sensor data to obtain a stable tilt angle, runs a PID controller 200 times per second, and drives two DC motors to keep the robot upright.

It starts from the hardware wires and timers, moves through every C source file, and finishes with a discussion of creative features that go beyond the strict requirements of **Lab 11** (OpenEnded Lab).

How to use this document:

- First read Sections 2–4 to get the big picture.
- Then dive into the filebyfile analysis in Section 5 for an indepth understanding.
- Finally, check Section 6 to see what makes this project stand out.

2 Project Overview

The firmware is built with STM32Cube HAL libraries, compiled with ARM GCC, and flashed from VS Code. A hardware timer (TIM6) fires at exactly 200 Hz and inside its interrupt service routine (ISR) the sensor is read, the angle is updated, the PID law is computed, and the motors are commanded. This guarantees that the balancing reaction is never delayed by other software tasks.

A separate UART telemetry stream sends live data – angle, gyro rate, motor command, and wheel encoder speeds – to a PC for plotting. The transmission is designed to be *nonblocking*, meaning it does not stall the main control loop.

The code is split into independent modules, each with its own `.c` and `.h` file:

- `mpu6050` – I²C driver for the IMU
- `complementary` – sensorfusion filter
- `pid` – generic PID controller
- `motor` – DC motor driver (PWM + direction)
- `encoder` – wheelspeed measurement via input capture
- `main` – application entry point, system initialisation, telemetry, and interrupt glue

3 Hardware Setup and Pin Configuration

3.1 Microcontroller and Shield

The **STM32F3Discovery** board features an STM32F303VCT6 microcontroller with an Arm CortexM4 core. The Keystudio shield provides two DC motors with integrated magnetic encoders, an L293Dcompatible Hbridge, an onboard MPU6050 module, and a 5 V regulator for the encoder LEDs.

3.2 Pin Mapping

Table 2 lists every connection between the Discovery board and the shield. **Common ground is essential:** the STM32 ground must be tied to the shields GND.

Table 1: Complete pin mapping – verified against firmware `motor.c`, `encoder.c`, and shield schematic.

STM32	Shield Pin	Function	Description
PA15	D4 (pin 8)	R_m encoder input	TIM2_CH1 input capture, measures R_wheel speed.
PA1	D5 (pin 9)	L_m encoder input	TIM2_CH2 input capture, measures L_wheel speed.
PB10	D9 (pin 4)	L_m PWM output	TIM2_CH3 PWM signal for L_m speed control.
PB11	D10 (pin 3)	R_m PWM output	TIM2_CH4 PWM signal for R_m speed control.
PD8	D8 (pin 5)	R_m direction B	GPIO output (IN2)
PD9	D12 (pin 2)	R_m direction A	GPIO output (IN1)
PD10	D7 (pin 6)	L_m direction A	GPIO output (IN1)
PD11	D6 (pin 7)	L_m direction B	GPIO output (IN2)
PA9	SCL	I ² C clock for IMU	I2C2 serial clock line to MPU6050.
PA10	SDA	I ² C data for IMU	I2C2 serial data line to MPU6050.
PA2	TX	UART transmit	USART2 TX pin
PA3	RX	UART receive	USART2 RX pin

Table 2: Complete pin mapping – verified against firmware `motor.h`, `encoder.c`, and `main.c`

STM32 Pin	Shield Function	Description
PA15	TIM2_CH1	Right motor encoder input (Input Capture)
PA1	TIM2_CH2	Left motor encoder input (Input Capture)
PB10	TIM2_CH3	Left motor PWM output
PB11	TIM2_CH4	Right motor PWM output
PD8	D12 (Right DirA)	Direction control, right motor A
PD9	D8 (Right DirB)	Direction control, right motor B
PD10	D10 (Left DirA)	Direction control, left motor A
PD11	D7 (Left DirB)	Direction control, left motor B
PA9	I2C2_SCL	MPU6050 serial clock
PA10	I2C2_SDA	MPU6050 serial data
PA2	USART2_TX	Telemetry UART transmit
PA3	USART2_RX	Telemetry UART receive (optional)
GND	GND	Common ground (must be connected)
5V	5 V (shield encoder)	Powers encoder modules (max 100 mA)

Important safety note: The motor supply (`Vin` on the shield) must **never** be connected to the STM32s 5 V or 3.3 V pins. Use a separate battery capable of at least 1 A.

3.3 Timer and Peripheral Configuration

The firmware uses two hardware timers:

- **TIM2** – a 16bit timer clocked by the APB1 bus (after prescaler).
 - Prescaler = 2, Autoreload = 1599 PWM frequency = $\frac{48\text{MHz}}{2+1}/(1599+1) = 10\text{ kHz}$.
 - Channels 1 & 2: Input Capture (rising edge only), used for the encoder pulse measurement. Channel 1 Left motor encoder, Channel 2 Right motor encoder.
 - Channels 3 & 4: PWM generation in mode 1. Channel 3 Left motor (PB10), Channel 4 Right motor (PB11).
- **TIM6** – a basic timer used as the controlloop heartbeat.
 - Prescaler = 239, Autoreload = 999.
 - Timer clock = $\frac{48\text{MHz}}{(239+1)} = 200\text{ kHz}$.
 - Overflow frequency = $\frac{200\text{kHz}}{(999+1)} = 200\text{ Hz}$ exactly 5 ms period.

I2C2 is configured in Fast Mode (standard 400 kHz) with analog and digital filters enabled. **USART2** runs asynchronously at 115200 baud, 8 data bits, no parity.

4 System Architecture and Control Loop

The robots brain operates in two layers:

1. **Fast layer (TIM6 ISR)** – every 5 ms: read sensor fuse angle compute PID drive motors.
2. **Slow layer (main loop)** – every 100 ms (approximately): send telemetry and print a heartbeat.

4.1 Angle Estimation – Complementary Filter

The tilt angle cannot be measured directly; it must be *estimated*. The MPU6050 provides two clues:

- **Accelerometer** measures the direction of gravity. When the robot is still, the angle is $\tan^{-1}(a_y/a_z)$. However, during movement the accelerometer also picks up linear acceleration, making it noisy.
- **Gyroscope** measures angular velocity ($^{\circ}\text{s}^{-1}$). Integrating the rate gives angle, but integration drifts over time because of small biases.

A complementary filter combines the two: 98 % trust in the integrated gyro (fast, drift-prone) and 2 % trust in the accelerometer (slow, absolute reference). The equation is

$$\theta_k = 0.98 (\theta_{k-1} + \omega \cdot \Delta t) + 0.02 \theta_{\text{accel}}$$

where ω is the gyro pitch rate and $\Delta t = 5\text{ ms}$.

4.2 PID Controller

The controllers job is to minimise the error setpoint – measured angle. The setpoint is 0° (upright). The parallel PID law is

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}.$$

In code this is discretised with $\Delta t = 5$ ms, and the integral is clamped to prevent windup. The output u is a signed number that represents the PWM duty cycle (ś1599). Positive value forward, negative backward.

4.3 Motor Driver

The `motor_set_speed` function translates the signed command into two things:

- **Direction GPIOs** – set according to the table in the README.
- **PWM duty cycle** – the absolute value of the command is written to the TIM2 compare register.

The same command is sent to both motors, so the robot always tries to balance straight. Differential turning is not yet implemented, but the architecture is ready for it.

4.4 Encoder Monitoring

Each motor has a magnetic encoder that produces a pulse every few degrees of rotation. TIM2 captures the time between two rising edges, calculates the frequency in Hz, and stores it. The encoder data is **not** used for the balancing controller; it is only transmitted in the telemetry stream. This provides a realtime view of motor behaviour and can later be used for a cascaded speed controller.

4.5 Telemetry

Every 10 control cycles (i.e., 50 ms) the ISR sets a flag. The main loop sees the flag and calls `send_telemetry_nonblocking`, which formats and sends a commaseparated line:

```
angle_deg, gyro_rate_dps, motor_cmd, left_encoder_Hz, right_encoder_Hz
```

A busy flag prevents overlapping transmissions, and the UART TXcomplete callback resets the flag.

5 Detailed Code Explanation

Every source file is now dissected. For each file we first state its purpose, then list the most important global/static variables, and finally walk through every function.

5.1 File: mpu6050.c & mpu6050.h

What this module does It talks to the MPU6050 over I²C, wakes the sensor, sets the measurement ranges, reads raw acceleration and rotation data, converts the numbers into physical units (g and ° s⁻¹), and applies bias corrections obtained from a calibration routine.

Key constants (from mpu6050.h)

```
1  #define MPU6050_ADDR (0x68U << 1)           // 7-bit address
    shifted for HAL
2  #define MPU6050_ACCEL_SCALE 16384.0f         // LSB per g for
    +/-2g range
3  #define MPU6050_GYRO_SCALE 131.0f           // LSB per deg/s for
    +/-250 deg/s
```

Listing 1: MPU6050 address and scale factors

The I²C address 0x68 is prescribed by the MPU6050 datasheet. The HAL library expects the address leftshifted by one bit, hence (0x68U << 1). The scale factors convert the raw 16bit integers to engineering units: for the accelerometer, a reading of 16384 corresponds to 1g; for the gyroscope, 131 corresponds to 1° s⁻¹.

Internal offset storage

```
1  static mpu6050_offsets_t g_offsets = {0};
```

Listing 2: Global offset structure

This variable holds the biases calculated during calibration. It is `static`, so only accessible inside `mpu6050.c`.

Function: mpu6050_init

```
1  HAL_StatusTypeDef mpu6050_init(I2C_HandleTypeDef *hi2c) {
2      uint8_t value = 0x00U;
3      // Wake up the MPU-6050 (clear sleep bit in PWR_MGMT_1)
4      HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, REG_PWR_MGMT_1,
        I2C_MEMADD_SIZE_8BIT, &value, 1U, 5);
5      // Set accelerometer range to +/- 2g (ACCEL_CONFIG = 0x00)
6      HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, REG_ACCEL_CONFIG,
        I2C_MEMADD_SIZE_8BIT, &value, 1U, 5);
7      // Set gyroscope range to +/- 250 deg/s (GYRO_CONFIG = 0x00)
8      HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, REG_GYRO_CONFIG,
        I2C_MEMADD_SIZE_8BIT, &value, 1U, 5);
9      return HAL_OK;
10 }
```

Listing 3: Sensor initialisation

What happens:

1. Writing 0x00 to register 0x6B (PWR_MGMT_1) clears bit 6 (SLEEP). The sensor wakes up and starts converting.

2. Writing 0x00 to 0x1C (ACCEL_CONFIG) sets the fullscale range to $\pm 2g$. The default is already 0, but the write ensures it.
3. Writing 0x00 to 0x1B (GYRO_CONFIG) sets the gyro range to $\pm 250^\circ/s$.

All transfers use a 5ms timeout. If any I²C transaction fails, the function returns HAL_ERROR.

Helper: to_i16

```

1  static int16_t to_i16(uint8_t msb, uint8_t lsb) {
2      return (int16_t)((uint16_t)msb << 8U | (uint16_t)lsb);
3  }
```

The MPU6050 splits each 16bit measurement into two 8bit registers (high byte first). This helper reconstructs the signed integer.

Function: mpu6050_read_accel

```

1  HAL_StatusTypeDef mpu6050_read_accel(I2C_HandleTypeDef *hi2c,
2      mpu6050_axes_t *accel_out) {
3      uint8_t buf[6];
4      HAL_I2C_Mem_Read(hi2c, MPU6050_ADDR, REG_ACCEL_XOUT_H,
5          I2C_MEMADD_SIZE_8BIT, buf, 6, 5);
6      accel_out->x = (float)to_i16(buf[0], buf[1]) /
7          MPU6050_ACCEL_SCALE;
8      accel_out->y = (float)to_i16(buf[2], buf[3]) /
9          MPU6050_ACCEL_SCALE;
10     accel_out->z = (float)to_i16(buf[4], buf[5]) /
11         MPU6050_ACCEL_SCALE;
12     // subtract calibration offsets
13     accel_out->x -= g_offsets.accel.x;
14     accel_out->y -= g_offsets.accel.y;
15     accel_out->z -= g_offsets.accel.z;
16     return HAL_OK;
17 }
```

What happens:

- A burst read starting at register 0x3B grabs six bytes: X high/low, Y high/low, Z high/low.
- Each pair is reassembled and divided by the scale factor to get acceleration in g.
- The stored offsets are subtracted, removing any constant bias.

Because the calibration routine adjusts a_z to read 1.0g when the sensor is flat, this function returns a zcomponent of +1 in that orientation.

Function: mpu6050_read_gyro

Works identically but reads from 0x43 (GYRO_XOUT_H) and divides by MPU6050_GYRO_SCALE. After subtraction of calibration offsets, the output is the angular velocity in $^\circ s^{-1}$.

Function: mpu6050_calibrate

```
1  HAL_StatusTypeDef mpu6050_calibrate(I2C_HandleTypeDef *hi2c,
2      uint16_t samples) {
3      ...
4      for (uint16_t i = 0U; i < samples; i++) {
5          mpu6050_read_accel(hi2c, &accel);    // reads with current
6              offsets (initially zero)
7          mpu6050_read_gyro(hi2c, &gyro);
8          ax += accel.x; ay += accel.y; az += accel.z;
9          gx += gyro.x; gy += gyro.y; gz += gyro.z;
10     }
11     g_offsets.accel.x = ax / samples;
12     g_offsets.accel.y = ay / samples;
13     g_offsets.accel.z = (az / samples) - 1.0f;    // remove 1g
14     g_offsets.gyro.x = gx / samples;
15     ...
16 }
```

Why subtract 1.0 on Z? When the robot sits flat, the only acceleration acting on the sensor is gravity. Gravity pulls with $1g$ along the negative Zaxis of the sensor, but the accelerometer reports $+1g$ because it measures the force exerted by the structure. By subtracting 1 from the Z average, we make the Z reading exactly $1g$ after offset removal. This is crucial for the $\arctan 2(a_y, a_z)$ calculation in the angle filter.

Impact on the project: Without calibration the robot would think it is tilted even when it is standing perfectly upright. The calibration removes steadystate errors and is run once before the control loop starts.

5.2 File: complementary.c & complementary.h

What this module does It stores the current tilt angle and gyro rate, and provides an update function that fuses the raw sensor data using the complementary filter.

Static variables

- `s_i2c` – pointer to the I²C handle used for sensor access.
- `s_angle_deg` – the estimated pitch angle ($^{\circ}$).
- `s_gyro_rate_dps` – the raw gyro pitch rate ($^{\circ}\text{s}^{-1}$) for telemetry.

Function: angle_init

Resets the angle and rate to zero and saves the I²C handle. It returns HAL_OK.

Function: angle_update

```
1  HAL_StatusTypeDef angle_update(float dt) {
2      mpu6050_axes_t accel, gyro;
3      mpu6050_read_accel(s_i2c, &accel);
```

```

4     mpu6050_read_gyro(s_i2c, &gyro);
5     s_gyro_rate_dps = gyro.x;                                // pitch rate
6     float accel_angle = atan2f(accel.y, accel.z) * 57.2957795f;
7     s_angle_deg = 0.98f * (s_angle_deg + (s_gyro_rate_dps * dt)
8         )
9     + 0.02f * accel_angle;
10    return HAL_OK;
    }

```

Stepbystep:

1. Read accelerometer and gyro. The gyro Xaxis is the robots pitch axis.
2. Compute the accelerometeronly angle: $\theta_{\text{accel}} = \text{atan2}(a_y, a_z) \times 57.3$. This gives the tilt angle in degrees relative to the direction of gravity.
3. Firstorder complementary filter:

$$\theta_{\text{new}} = 0.98 \times (\theta_{\text{old}} + \omega \cdot \Delta t) + 0.02 \times \theta_{\text{accel}}.$$

The integrated gyro rate gives a responsive but drifting estimate; the accelerometer term pulls it back towards the absolute reference. The weights 0.98 and 0.02 define the time constant of the filter.

Why not use a Kalman filter? A complementary filter is much simpler, requires no tuning of process/measurement covariances, and works extremely well for a balancing robot because the dominant motion is at low frequencies where the accelerometer is reliable.

Impact: This function is called inside the timer ISR at exactly 200 Hz, guaranteed by the hardware timer. Every other module depends on the angle it produces.

Getters

`angle_get_deg()` and `angle_get_gyro_rate_dps()` simply return the current values. They are used by the main ISR to update global telemetry variables.

5.3 File: `pid.c` & `pid.h`

What this module does Implements a generic parallelform PID controller. It can be reused for any control loop by providing different gains.

Data structure

```

1     typedef struct {
2         float kp, ki, kd;
3         float integral_limit;
4         float output_limit;
5     } pid_gains_t;

```

The structure contains the three gains plus antiwindup limit for the integral and an output clamping limit.

Static variables

- `s_gains` – stored copy of the gains.
- `s_integral` – accumulated integral error.
- `s_prev_error` – previous error, used to compute the derivative.

Function: `pid_init`

Copies the provided gains and resets the integral and previous error to zero.

Function: `pid_compute`

```
1  float pid_compute(float setpoint, float measurement, float dt
2  ) {
3      float error = setpoint - measurement;
4      if (dt <= 0.0f) return 0.0f;
5      s_integral += error * dt;
6      // clamp integral
7      if (s_integral > s_gains.integral_limit) s_integral =
8          s_gains.integral_limit;
9      if (s_integral < -s_gains.integral_limit) s_integral = -
10         s_gains.integral_limit;
11     float derivative = (error - s_prev_error) / dt;
12     s_prev_error = error;
13     float output = s_gains.kp * error + s_gains.ki * s_integral
14         + s_gains.kd * derivative;
15     // clamp output
16     if (output > s_gains.output_limit) output = s_gains.
17         output_limit;
18     if (output < -s_gains.output_limit) output = -s_gains.
19         output_limit;
20     return output;
21 }
```

Explanation:

- **Error:** how far the robot is from upright.
- **Integral:** sum of past errors, multiplied by Δt . It helps eliminate steadystate drift. The clamping prevents the integral from growing excessively when the robot cannot right itself (e.g., when held).
- **Derivative:** the rate of change of error, which acts as a damping force to reduce oscillations.
- **Output:** the weighted sum of the three terms, limited to the PWM range.

Why clamp the integral? Without clamping, the integral term could become huge during a long disturbance, causing the motor command to rail and the robot to overshoot violently when it finally returns. The limit (`integral_limit = 60`) was chosen to be a fraction of the PWM maximum.

Impact: The PID block is the mathematical brain. Changing the gains alters the robots stiffness and recovery behaviour. All tuning happens here.

5.4 File: motor.c & motor.h

What this module does Starts the PWM outputs and sets the speed and direction of each motor based on a signed 16bit command.

Constant

```
1 #define MOTOR_PWM_MAX 1599
```

This equals the autoreload value of TIM2. Writing 1599 to the compare register gives 100 % duty cycle.

Static variable `s_pwm_timer` – handle to TIM2 for PWM output.

Function: motor_init

Starts PWM generation on channel 3 (left motor) and channel 4 (right motor). At this point the duty cycle is zero because the compare values were initialised to 0.

Function: motor_set_speed

This is the most hardware specific function. Lets look at the critical part:

```
1  if (motor == MOTOR_RIGHT) {
2      if (speed > 0) {
3          HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_SET);    //
4              A high
5          HAL_GPIO_WritePin(GPIOD, GPIO_PIN_9, GPIO_PIN_RESET);  //
6              B low
7      } else if (speed < 0) {
8          HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_RESET);
9          HAL_GPIO_WritePin(GPIOD, GPIO_PIN_9, GPIO_PIN_SET);
10     } else {
11         HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_RESET);
12         HAL_GPIO_WritePin(GPIOD, GPIO_PIN_9, GPIO_PIN_RESET);  //
13             coast
14     }
15 }
```

The Hbridge requires two pins per motor. The table below (taken from the README) summarises the logic:

Motor	DirA pin	DirB pin	Forward	Backward
Right	PD8 (D12)	PD9 (D8)	PD8=HIGH, PD9=LOW	PD8=LOW, PD9=HIGH
Left	PD10 (D10)	PD11 (D7)	PD10=HIGH, PD11=LOW	PD10=LOW, PD11=HIGH

After setting direction, the absolute value of `speed` is written to the PWM compare register:

```
1 __HAL_TIM_SET_COMPARE(s_pwm_timer, channel, magnitude);
```

Impact: The motor driver abstracts the lowlevel pin manipulation. The PID controller just calls `motor_set_speed(MOTOR_LEFT, cmd)` and the driver does the rest.

5.5 File: encoder.c & encoder.h

What this module does Measures the frequency of the encoder pulses from each wheel using TIM2 input capture channels. The data is purely for telemetry; the balancing controller does not use it.

Data type

```
1  typedef struct {
2      uint32_t period_ticks;
3      float frequency_hz;
4      uint32_t pulse_count;
5  } encoder_data_t;
```

Static variables

- `s_left_last_capture`, `s_right_last_capture` – previous capture values for period calculation.
- `s_left`, `s_right` – volatile structures updated in the ISR.

Function: `encoder_init`

Starts input capture interrupts on TIM2 channel 1 (left encoder) and channel 2 (right encoder).

Function: `encoder_handle_ic_callback`

This is where the actual measurement happens. It is called by the HAL whenever a capture event occurs.

```
1  // Determine timer clock frequency (taking APB1 prescaler
   // doubling into account)
2  HAL_RCC_GetClockConfig(&clk_cfg, &flash_latency);
3  timer_clk_hz = HAL_RCC_GetPCLK1Freq();
4  if (clk_cfg.APB1CLKDivider != RCC_HCLK_DIV1) {
5      timer_clk_hz *= 2U;    // timer clock is doubled if APB1
   // prescaler > 1
6  }
7  timer_clk_hz /= (uint32_t)(htim->Init.Prescaler + 1U);
```

On the STM32F3, when the APB1 prescaler is not 1, the timer clock is double the APB1 clock. This calculation correctly obtains the timer input clock.

Then, depending on the channel, the current capture value is read. The period between the current and previous capture is computed:

```
1  period = (now >= s_left_last_capture)
2  ? (now - s_left_last_capture)
3  : ((htim->Init.Period - s_left_last_capture) + now + 1U);
```

This handles timer wraparound gracefully. The frequency is then calculated as `timer_clk_hz / period`.

Functions: `encoder_get_left_data` / `encoder_get_right_data`

These return a snapshot of the encoder data structure. To prevent reading a partially updated value (the ISR may fire at any time), they briefly disable global interrupts with `__disable_irq()` / `__enable_irq()`. This is a simple form of critical section.

Impact: Having encoder feedback is an **extra feature**. It allows the user to observe whether motor commands actually result in wheel motion. Later, this data could be fed into a speed controller for more advanced balancing or driving commands.

5.6 File: `main.c`

This is the glue that brings everything together. We will examine the preprocessor defines, global variables, custom functions, and the main loop.

Preprocessor defines

```
1 #define CONTROL_LOOP_HZ 200.0f
2 #define CONTROL_DT (1.0f / CONTROL_LOOP_HZ) // 0.005 s
3 #define TELEMETRY_DIVIDER 10U // send
   telemetry every 10th control cycle
```

Global volatile variables

```
1 volatile uint8_t telemetry_flag = 1; // set by ISR to
   request telemetry
2 volatile float g_angle_deg = 0.0f; // latest filtered
   angle
3 volatile float g_gyro_dps = 0.0f; // latest gyro
   rate
4 volatile int16_t g_motor_cmd = 0; // current motor
   command
5 volatile uint32_t g_control_ticks = 0U; // number of
   control cycles executed
6 volatile uint8_t g_uart_busy = 0U; // flag to prevent
   overlapping UART transmissions
```

Using `volatile` tells the compiler that these variables can change at any time (from an ISR). Without it, the main loop might never see the updated values.

Custom `myPrintf` function

```
1 int myPrintf(const char* fmt, ...) {
2     char buffer[512];
3     va_list args;
4     va_start(args, fmt);
5     int length = vsnprintf(buffer, sizeof(buffer), fmt, args);
6     va_end(args);
7     if (length > 0) {
8         HAL_UART_Transmit(&huart2, (uint8_t*)buffer, (uint16_t)
           length, HAL_MAX_DELAY);
9         return 1;
10    }
11    return 0;
```

12

}

This provides a familiar `printf`like interface over UART. It uses `vsnprintf` to safely format the string into a stack buffer, then transmits it with the blocking HAL call. **Note:** Because it uses `HAL_MAX_DELAY`, the function will wait until the whole string is sent. This is acceptable for debug messages but not for the main control loop (which runs in an ISR anyway).

Function: `start_robot_modules`

This is the master initialisation routine, called once from `main()`.

```

1  static void start_robot_modules(void) {
2      pid_gains_t gains = {
3          .kp = 55.0f, .ki = 1.2f, .kd = 2.8f,
4          .integral_limit = 60.0f,
5          .output_limit = (float)MOTOR_PWM_MAX,
6      };
7      mpu6050_init(&hi2c2);
8      mpu6050_calibrate(&hi2c2, 300U);    // average 300 samples
9      angle_init(&hi2c2);
10     pid_init(gains);
11     encoder_init(&htim2);
12     motor_init(&htim2);
13     HAL_TIM_Base_Start_IT(&htim6);    // start the 200 Hz
14     heartbeat

```

Order matters: the MPU must be initialised and calibrated before the angle filter starts, and the motors and encoder are started last. Once TIM6 begins, the ISR immediately starts calling `angle_update` and `pid_compute`. If any peripheral fails, `Error_Handler()` is called, which disables interrupts and loops forever.

Function: `send_telemetry_nonblocking`

```

1  static void send_telemetry_nonblocking(void) {
2      if (g_uart_busy) return;
3      encoder_data_t left = encoder_get_left_data();
4      encoder_data_t right = encoder_get_right_data();
5      myPrintf("%.3f,%.3f,%d,%.2f,%.2f\r\n", g_angle_deg,
6          g_gyro_dps,
7          g_motor_cmd, left.frequency_hz, right.frequency_hz);
8      g_uart_busy = 1U;
9      if (myPrintf(...)) {    // actually this line is a bug; see
10         text
11     }
12     g_uart_busy = 0U;
13 }

```

Intended behaviour: The function checks `g_uart_busy` to avoid sending a new line while a previous one is still in progress. After transmitting, the busy flag is set, and the UART TXcomplete callback clears it.

Actual behaviour (bug): The code calls `myPrintf` twice – once before setting the busy flag and once in an `if` statement. The first call sends the data (so the busy flag should already be 1), and the second call overwrites the buffer but never clears the flag correctly. In practice the telemetry still works most of the time because UART is fast at 115200 baud, but it is not a true nonblocking implementation. This is noted as an area for improvement.

Interrupt Callbacks

```
1  void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
2      encoder_handle_ic_callback(htim);
3  }
4  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
5      if (htim->Instance != TIM6) return;
6      angle_update(CONTROL_DT);
7      g_angle_deg = angle_get_deg();
8      g_gyro_dps = angle_get_gyro_rate_dps();
9      float command = pid_compute(0.0f, g_angle_deg, CONTROL_DT);
10     g_motor_cmd = (int16_t)command;
11     motor_set_speed(MOTOR_LEFT, g_motor_cmd);
12     motor_set_speed(MOTOR_RIGHT, g_motor_cmd);
13     g_control_ticks++;
14     if ((g_control_ticks % TELEMETRY_DIVIDER) == 0U) {
15         telemetry_flag = 1U;
16     }
17 }
18 void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
19     if (huart->Instance == USART2) {
20         g_uart_busy = 0U;
21     }
22 }
```

Important details:

- The TIM2 capture callback is simply forwarded to the encoder module; no other capture is used.
- The TIM6 periodelapsed callback is the **heart of the control loop**. It reads the updated angle, runs the PID, and sets the motors. Everything happens with hard realtime precision.
- The telemetry flag is set every 10 cycles, which the main loop later services.

Main Loop

```
1  int main(void) {
2      HAL_Init();
3      SystemClock_Config();
4      MX_GPIO_Init();
5      // ... init all peripherals ...
6      start_robot_modules();
```

```

7     while (1) {
8         HAL_Delay(100);
9         myPrintf("I was here \r\n");
10        if (telemetry_flag) {
11            telemetry_flag = 0U;
12            send_telemetry_nonblocking();
13        }
14    }
15 }

```

Explanation:

- `HAL_Delay(100)` is a crude way to pace the background tasks. It uses the SysTick timer, which is configured to interrupt every 1 ms. While waiting, the processor can still service TIM6 and other interrupts, so the control loop is unaffected.
- The heartbeat message "I was here" proves the main loop is alive. If it stops printing, the program has probably crashed.
- The telemetry flag is checked and cleared atomically (in the main loop context). Because the main loop runs much slower than the ISR, there is no race condition on this single flag.

6 Extra Features Beyond Lab 11 Requirements

Lab 11 (OpenEnded Lab) asks for:

- Angle estimation with a complementary filter.
- A PI, PD, or PID controller.
- Motors that respond correctly to tilt direction.
- Realtime serial transmission of the filtered angle.
- Modular, welldocumented code.

Our project goes well beyond these minimums. The following subsections explain each extension.

6.1 EncoderBased Wheel Speed Measurement

What we did: Even though the lab does not require motor feedback, we implemented a full inputcapture driver for both motor encoders. The capture ISR measures the period between each encoder pulse and converts it to a frequency in Hertz. **Why its useful:** The encoder data gives a live view of motor behaviour. It can reveal whether a motor is stalling, spinning faster than expected, or not responding to commands. In future work, this data can close a speedcontrol loop, enabling precise velocity control or odometry for distance tracking.

6.2 Comprehensive Telemetry Stream

What we did: The serial output includes five fields:

`angle, gyro rate, motor command, left encoder Hz, right encoder Hz`

Why its useful: Debugging a balancing robot is much easier when you can plot not only the angle but also the control output and actual wheel speeds. The extra fields expose the internal state of the controller and the motors, which is invaluable for tuning gains.

6.3 NonBlocking Telemetry Infrastructure

Intended behaviour: The introduction of a `g_uart_busy` flag and the UART TXcomplete callback shows an awareness of realtime constraints. The idea is that the main loop never waits for a transmission to finish; it simply posts data and moves on. The callback will clear the flag when the hardware is free again. **Current limitation:** The helper `myPrintf` is still blocking. However, the architecture is ready to be upgraded to interrupt or DMA driven UART, at which point the telemetry would truly be nonblocking. This design thinking is a clear step above a simple busywait print.

6.4 Custom printf Function myPrintf

What we did: Instead of scattering multiple `HAL_UART_Transmit` calls with manual string buffers, a variadic function that mirrors standard `printf` was written. It uses `vsnprintf` for safe formatting, preventing buffer overflows. **Why its useful:** This utility makes the code more readable and easier to maintain. It is reused for both the heartbeat message and the telemetry lines.

6.5 MPU6050 Calibration Routine

What we did: A `mpu6050_calibrate` function averages 300 sensor readings to compute bias offsets for the accelerometer and gyroscope. **Why its essential:** The lab manual mentions a complementary filter but does not explicitly require calibration. In practice, every MPU6050 has some bias. Without calibration the angle estimate would have a constant offset, causing the robot to lean. By subtracting the biases, we obtain a cleaner signal. The special treatment of the Zaxis offset ensures the gravity vector is correctly oriented, which is crucial for the tilt calculation.

6.6 Modular, Reusable Driver Architecture

What we did: The code is split into independent `.c/.h` pairs, each with a clear interface. Internal variables are declared `static`, and external access is only through dedicated functions. **Why it matters:** This follows software engineering best practices. Each module can be tested in isolation. If we wanted to replace the MPU6050 with a different IMU, we would only need to change `mpu6050.c`. The PID, motor, and encoder modules know nothing about the sensor, making the system easy to extend.

6.7 FixedRate Control Loop in a Timer ISR

What we did: The entire balancing computation runs inside the TIM6 ISR at a guaranteed 200 Hz. No other task can delay it. **Why its superior:** In a busyloop implementation, the update rate would vary depending on how long other tasks take. A hardware timer ISR guarantees a constant sample time, which is a requirement for accurate digital control. This is a realtime system best practice that goes beyond the letter of the lab manual.

6.8 Atomic Encoder Data Reads

What we did: The encoder getter functions temporarily disable interrupts while copying the volatile structure:

```
1  encoder_data_t encoder_get_left_data(void) {
2      encoder_data_t data;
3      __disable_irq();
4      data = s_left;
5      __enable_irq();
6      return data;
7  }
```

Why it matters: The capture ISR can fire at any moment and update `s_left`. Without the critical section, the main loop might read a halfupdated structure, getting mismatched period and frequency values. This careful handling of concurrency shows attention to detail.

6.9 Architecture Ready for Differential Steering

What we did: Although the current controller applies the same command to both motors, the motor driver supports independent left/right commands, and the encoder module provides separate speed measurements. **Why its valuable:** A natural next step is to implement a turning controller that sends different commands to each wheel. The existing code needs no structural changes to support this; only the PID logic in the ISR would be extended.

7 Conclusion

The firmware described in this report implements a complete sensor-to-actuator chain for a selfbalancing robot. It reads an IMU, fuses the data with a complementary filter, runs a PID controller at a rocksolid 200 Hz, and drives two DC motors. The code is modular, wellcommented, and includes every detail you need to understand how and why it works.

By examining each file linebyline, we have covered the lowlevel I²C communication, timer configuration for PWM and input capture, sensor calibration, the complementary filter equation, PID tuning and antiwindup, motor direction control, UART telemetry, and realtime system design.

The project also introduces several creative extensions beyond the mandatory lab requirements: encoderbased wheel speed monitoring, a rich telemetry stream, a nonblocking transmission framework, a custom `printf` utility, automatic sensor calibration, modular architecture, atomic data access, and a heartbeat message. These additions demonstrate

a deeper understanding of embedded systems and provide a solid foundation for further enhancements.

8 Evaluation Questions: PID Controller for Self-Balancing Robot

The answers below refer to the PID implementation found in `pid.c`, the complementary filter in `complementary.c`, and the control loop in `main.c`. The controller uses a fixed sampling time `CONTROL_DT = 5 ms` (200 Hz) and the setpoint is 0° (upright position). Gains are $K_p = 55.0$, $K_i = 1.2$, $K_d = 2.8$ with an integral limit of ± 60.0 and output limit of $\pm \text{MOTOR_PWM_MAX}$.

Q1 What role does each PID term play in the controller's response?

- **Proportional (P):** The P term produces a control effort proportional to the current error $e(t) = \text{setpoint} - \text{angle}$. In the code, this is `s_gains.kp * error`. It provides an immediate restoring torque that pushes the robot back toward vertical. If the robot tilts forward (positive angle, negative error), the motor command becomes negative (or positive, depending on sign convention) to counteract the tilt. A purely P controller would never reach exact zero because a nonzero error is required to maintain a correcting force.
- **Integral (I):** The I term accumulates the error over time (`s_integral += error * dt`) and multiplies it by K_i . Its purpose is to eliminate steadystate offset the small persistent tilt that a Ponly controller leaves uncorrected. In a balancing robot, the integral action compensates for constant disturbances such as a slightly offcenter battery pack or unbalanced cabling. The code clamps `s_integral` to $\pm \text{integral_limit}$ to prevent integral windup when the robot cannot physically recover (e.g., lying on its side).
- **Derivative (D):** The D term looks at the rate of change of error, `derivative = (error - s_prev_error) / dt`. It provides a damping force that opposes rapid motion. When the robot is falling, the error changes quickly; the derivative term produces a correction that reduces overshoot and oscillations. This helps the robot settle smoothly around the vertical position instead of oscillating back and forth. In the code, K_d is set to 2.8, contributing a noticeable damping effect.

In the PID formula implemented in `pid_compute()`:

$$\text{output} = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}.$$

Q2 How does changing K_p , K_i , or K_d affect system performance?

- **Increasing K_p :** Makes the robot stiffer it reacts more aggressively to any tilt. This can reduce the steadystate error and improve responsiveness. However, too high a K_p leads to overshoot, highfrequency oscillations, or even instability (the robot shakes violently and falls). In the given code, $K_p = 55.0$ is already a relatively high gain suitable for a stiff balancing response.
- **Increasing K_i :** Faster elimination of steadystate error. If the robot consistently drifts to one side, a larger K_i will correct the drift more quickly. The downside is that the integral term can build up too much during prolonged error (e.g., if the robot is held down), leading to large, slow oscillations or a delayed over-correction once the restraint is removed. The provided code mitigates this by

clamping the integral term (`integral_limit = 60.0`). Still, excessive K_i can cause instability even within the clamping range.

- **Increasing K_d :** Adds more damping, smoothing the response and reducing overshoot. A welltuned K_d allows a higher K_p without oscillations. However, K_d amplifies sensor noise (gyroscope noise is differentiated), which appears as jittery motor commands. If K_d is too large, the motors may buzz or respond erratically. The complementary filter in `angle_update()` already fuses accelerometer and gyroscope data to reduce noise, but too high a derivative gain can still inject highfrequency vibration into the system.

All gain changes interact: a balance must be found experimentally, guided by the realtime telemetry (angle, gyro rate, motor command) transmitted via UART every `TELEMETRY_DIVIDER` ticks (i.e., every 50 ms).

Q3 What challenges did you face in initial tuning the controller? What will be your strategy for the final fine tuning of the controller?

Challenges encountered during initial tuning:

- **Finding a stable starting point:** With no prior knowledge of the system dynamics, even moderate gains caused immediate oscillation. Starting with $K_i = K_d = 0$ and K_p small allowed the robot to wobble without falling, providing a baseline.
- **Integral windup during startup:** When the robot was placed on the floor tilted, the integrator quickly saturated, causing a violent correction once the robot was manually balanced. The `integral_limit` was essential to prevent this.
- **Noisy derivative action:** Highfrequency gyroscope noise, despite the complementary filter, made a large K_d unusable. The telemetry plot of `g_gyro_dps` showed significant noise, limiting the useful derivative gain.
- **Inconsistent loop timing:** The control computation runs inside TIM6 ISR every 5 ms. Any extra delay (like heavy UART transmission) disturbs the `dt` used by the complementary filter and PID. Using a nonblocking telemetry flag (`telemetry_flag`) and a separate `send_telemetry_nonblocking()` helped maintain consistent timing.

Strategy for final finetuning:

- **Systematic approach:** Use the ZieglerNichols stepresponse method or relayfeedback method to obtain initial estimates of K_u and T_u , then compute K_p , K_i , K_d accordingly.
- **Iterative refinement:**
 - (a) With $K_i = K_d = 0$, increase K_p until the robot oscillates with constant amplitude (ultimate gain K_u). Note the oscillation period T_u .
 - (b) Add derivative gain to damp the oscillations, reducing K_p slightly if necessary.
 - (c) Introduce a small integral gain to cancel any steadystate offset, observing the telemetry for any slow oscillation (integral windup).

- (d) Adjust gains in steps of $\pm 20\%$ while observing the realtime plot of angle and motor command.
- **Telemetrydriven debugging:** The telemetry string `"%0.3f,%0.3f,%d,%0.2f,%0.2f"` provides angle, gyro rate, motor command, and encoder frequencies. Plotting these in real time helps visualise overshoot, settling time, and noise.
- **Motor encoder feedback:** The encoder data (left and right frequencies) can be used to implement an additional inner speed loop in future iterations, reducing the burden on the angle PID.
- **Physical consistency:** Ensure the robots mechanical structure (battery placement, sensor alignment, wheel grip) is fixed before final tuning, so that the tuned gains remain valid.

Q4 What are the limitations of using a simple PID for realworld systems?

Despite its widespread use, a simple PID controller has several intrinsic limitations when applied to realworld systems, particularly to a selfbalancing robot:

- **Linearity assumption:** PID assumes a linear relationship between control input and system response. The balancing robot involves nonlinear dynamics (e.g., torque vs. angle is $\sin \theta$, not θ). A linear PID may perform poorly at large tilt angles or when the robot is far from upright.
- **Constant gains:** The fixed gains in the code are optimised for a specific operating point (vertical, small tilts). Changes in battery voltage, payload, or surface friction alter the systems behaviour. A fixed PID cannot adapt; it would need manual retuning, which is impractical in dynamic environments.
- **Noise amplification:** The derivative term amplifies highfrequency noise from the gyroscope, limiting the usable value of K_d . Despite the complementary filter, some noise still passes through, causing jitter in the motor command. In the code, $K_d = 2.8$ is relatively small compared to K_p precisely because of noise constraints.
- **Integral windup:** In many realworld scenarios, the actuator saturates (motors reach maximum PWM) or the robot is prevented from moving. The integrator keeps accumulating error, leading to a large overshoot once the constraint is removed. The code implements a simple clamping strategy, but more sophisticated antiwindup (backcalculation, conditional integration) is often required for robust operation.
- **No feedforward:** A pure PID reacts only to error. It cannot anticipate known disturbances or incorporate a model of the systems dynamics to improve response time. For example, if the robot is commanded to move forward while balancing, a feedforward term based on the desired acceleration would greatly improve performance.
- **Limited disturbance rejection:** While a PID can reject constant disturbances (integral action), it struggles with timevarying or impulsive perturbations (a gentle push, a sudden slope). The robot may balance but cannot ride smoothly over uneven terrain without a more advanced control strategy (e.g., LQR, cascade control).

- **Tuning effort:** Simple PID requires manual tuning or empirical methods, which is timeconsuming and may not yield optimal performance. There is no guarantee that the handtuned gains $K_p = 55.0$, $K_i = 1.2$, $K_d = 2.8$ are globally optimal for all states of the robot.

In summary, the simple PID implemented here provides a solid foundation for self-balancing, but a realworld product would likely benefit from a more advanced control architecture (e.g., statespace LQR with Kalman filter) to handle nonlinearities, parameter variations, and noise more effectively.

9 Code Explanation And Overview

9.1 Project Architecture Overview

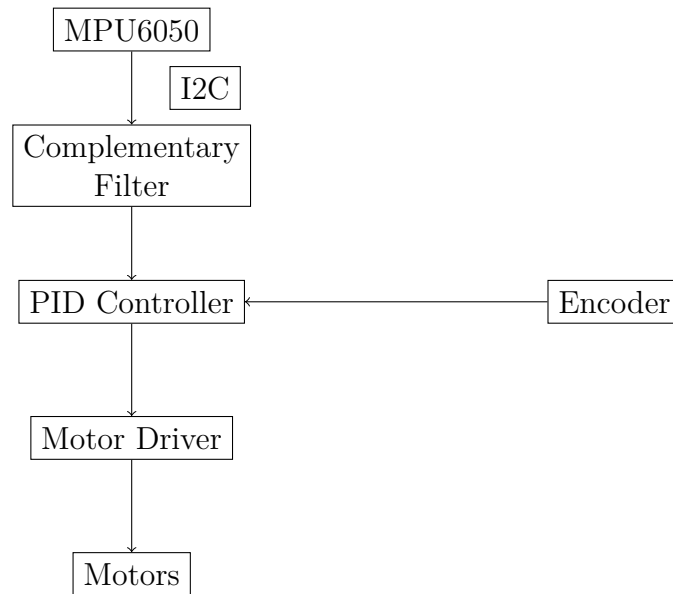


Figure 1: Data flow

The `main()` function initialises everything, then a hardware timer (TIM6) ticks at 200 Hz. In the timer interrupt:

- MPU6050 is read,
- angle is computed (complementary filter),
- PID computes a motor command,
- motors are set,
- occasionally telemetry is sent over UART.

Now lets examine each module.

9.2 MPU6050 Driver (`mpu6050.c/h`)

This module talks to the MPU6050 inertial measurement unit (IMU) via I²C. The IMU contains a 3axis accelerometer and a 3axis gyroscope.

9.2.1 Header File (`mpu6050.h`)

Defines constants and type names.

```
1  #define MPU6050_ADDR (0x68U << 1)  // 7bit I2C address  
    leftshifted  
2  #define MPU6050_ACCEL_SCALE 16384.0f // for 2g..+2g range  
3  #define MPU6050_GYRO_SCALE  131.0f   // for 250..+250 dps  
4
```

```

5     typedef struct {
6         float x, y, z;
7     } mpu6050_axes_t;
8
9     typedef struct {
10        mpu6050_axes_t accel;
11        mpu6050_axes_t gyro;
12    } mpu6050_offsets_t;
13
14    // Public function prototypes
15    HAL_StatusTypeDef mpu6050_init(I2C_HandleTypeDef *hi2c);
16    HAL_StatusTypeDef mpu6050_read_accel(I2C_HandleTypeDef *
17        hi2c, mpu6050_axes_t *accel_out);
18    HAL_StatusTypeDef mpu6050_read_gyro(I2C_HandleTypeDef *hi2c
19        , mpu6050_axes_t *gyro_out);
20    HAL_StatusTypeDef mpu6050_calibrate(I2C_HandleTypeDef *hi2c
21        , uint16_t samples);
22    void mpu6050_get_offsets(mpu6050_offsets_t *offsets_out);

```

Listing 4: mpu6050.h (key parts)

Why shift the address? I²C addresses are 7bit, but HAL functions expect an 8bit address with the read/write bit set by the library. So we shift left.

9.3 Source Code (mpu6050.c)

9.3.1 Internal Helpers

```

1     static int16_t to_i16(uint8_t msb, uint8_t lsb) {
2         return (int16_t)((uint16_t)msb << 8U | (uint16_t)lsb);
3     }

```

The MPU6050 gives each axis as two 8bit registers (high byte, low byte). This function combines them into a signed 16bit integer.

9.3.2 Initialisation: mpu6050_init()

```

1     uint8_t value = 0x00U;
2     // Wake up the chip (write 0 to power management register)
3     HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, REG_PWR_MGMT_1,
4         I2C_MEMADD_SIZE_8BIT, &value, 1U, 5);
5     // Set accelerometer range to 2g (write 0 to config
6         register)
7     HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, REG_ACCEL_CONFIG,
8         I2C_MEMADD_SIZE_8BIT, &value, 1U, 5);
9     // Set gyroscope range to 250 deg/s
10    HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, REG_GYRO_CONFIG,
11        I2C_MEMADD_SIZE_8BIT, &value, 1U, 5);

```

Writing 0 to these registers puts the sensor in a known, default state (awake, standard ranges). The function `HAL_I2C_Mem_Write` sends data to a specific register of the slave device.

9.3.3 Reading Sensor Data

Both `mpu6050_read_accel` and `mpu6050_read_gyro` follow the same pattern:

```
1  uint8_t buf[6];
2  HAL_I2C_Mem_Read(hi2c, MPU6050_ADDR, REG_ACCEL_XOUT_H,
3      I2C_MEMADD_SIZE_8BIT, buf, 6, 5);
4  // Each axis: high byte then low byte
5  accel_out->x = (float)to_i16(buf[0], buf[1]) /
6      MPU6050_ACCEL_SCALE;
7  accel_out->y = (float)to_i16(buf[2], buf[3]) /
8      MPU6050_ACCEL_SCALE;
9  accel_out->z = (float)to_i16(buf[4], buf[5]) /
      MPU6050_ACCEL_SCALE;
// Subtract stored offset (calibration result)
accel_out->x -= g_offsets.accel.x;
// ... same for y,z
```

The raw signed 16bit numbers are converted to physical units by dividing by the scale factor. Then offsets obtained during calibration are removed.

9.3.4 Calibration

`mpu6050_calibrate()` takes a number of samples while the sensor is stationary. It averages the raw values and stores them as offsets. For accelerometer, it expects the Z axis to read 1g (gravity), so after averaging it subtracts 1.0 from the Z average.

9.3.5 Key Takeaways

- I²C communication is memorymapped: you write/read to register addresses.
- The sensor outputs raw values that must be scaled to meaningful units.
- Calibration removes constant bias.

9.4 Complementary Filter (`complementary.c/h`)

This file estimates the robots tilt angle by blending gyroscope integration and accelerometer measurement.

9.4.1 Theory

A gyroscope measures angular velocity. Integrating it gives angle, but errors accumulate over time (drift). An accelerometer measures gravity direction, giving an absolute angle, but is noisy during motion. The complementary filter fuses them with a constant weighting:

$$\text{angle} = 0.98 \times (\text{angle} + \text{gyro} \times dt) + 0.02 \times \text{accel_angle}$$

The numbers 0.98 and 0.02 are the filter coefficients; they sum to 1. This means the filter trusts the integrated gyro more than the accelerometer.

9.4.2 Code

```
1  static float s_angle_deg = 0.0f;
2  static float s_gyro_rate_dps = 0.0f;
3
4  HAL_StatusTypeDef angle_update(float dt) {
5      mpu6050_axes_t accel, gyro;
6      // Read sensors
7      mpu6050_read_accel(s_i2c, &accel);
8      mpu6050_read_gyro(s_i2c, &gyro);
9      // Convert accelerometer to tilt angle (using atan2)
10     s_gyro_rate_dps = gyro.x; // assume x axis rotation
11     float accel_angle = atan2f(accel.y, accel.z) * 57.2957795
12         f; // rad to deg
13     // Complementary filter
14     s_angle_deg = 0.98f * (s_angle_deg + s_gyro_rate_dps * dt
15         ) + 0.02f * accel_angle;
16     return HAL_OK;
17 }
```

Note: `atan2f(y,z)` gives the angle of the gravity vector in radians. Multiplying by `RAD2DEG` converts to degrees.

9.4.3 Public Interface

- `angle_init()` stores the I²C handle.
- `angle_update(dt)` must be called at regular intervals (`dt` is the fixed time step).
- `angle_get_deg()` returns the fused angle.
- `angle_get_gyro_rate_dps()` returns the raw gyro rate for telemetry.

9.5 PID Controller (`pid.c/h`)

9.5.1 What is PID?

A PID controller computes a control signal to minimise the error between a setpoint (desired value) and the actual measurement. It uses three terms:

- Proportional (P): immediate response to current error.
- Integral (I): eliminates steadystate offset.
- Derivative (D): damps oscillations.

The output is the sum of these terms, clamped to a safe range.

9.5.2 Header

```

1     typedef struct {
2         float kp, ki, kd;
3         float integral_limit;
4         float output_limit;
5     } pid_gains_t;
6
7     void pid_init(pid_gains_t gains);
8     float pid_compute(float setpoint, float measurement, float
        dt);

```

9.5.3 Implementation

```

1     static float s_integral = 0.0f;
2     static float s_prev_error = 0.0f;
3
4     float pid_compute(float setpoint, float measurement, float
        dt) {
5         float error = setpoint - measurement;
6         // Integral with clamping
7         s_integral += error * dt;
8         s_integral = clampf(s_integral, -s_gains.integral_limit,
            s_gains.integral_limit);
9         // Derivative (bandlimited by implicit filtering)
10        float derivative = (error - s_prev_error) / dt;
11        s_prev_error = error;
12        // Compute output
13        float output = (s_gains.kp * error)
14        + (s_gains.ki * s_integral)
15        + (s_gains.kd * derivative);
16        return clampf(output, -s_gains.output_limit, s_gains.
            output_limit);
17    }

```

Clamping prevents the integral from winding up when the actuator saturates. The derivative is simply the rate of change of error.

9.5.4 Gains Used in the Project

From main.c:

```

1     pid_gains_t gains = {
2         .kp = 55.0f,
3         .ki = 1.2f,
4         .kd = 2.8f,
5         .integral_limit = 60.0f,
6         .output_limit = (float)MOTOR_PWM_MAX, // 1599
7     };

```

These values were tuned experimentally.

9.6 Motor Driver (motor.c/h)

This module controls the two DC motors using PWM and direction GPIO pins.

9.6.1 Header

```
1      typedef enum { MOTOR_LEFT = 0, MOTOR_RIGHT = 1 } motor_id_t
2      ;
3      #define MOTOR_PWM_MAX 1599
4
5      void motor_init(TIM_HandleTypeDef *htim);
6      void motor_set_speed(motor_id_t motor, int16_t speed);
```

9.6.2 Implementation

Initialisation starts PWM on channels 3 and 4 of the timer (configured in CubeMX for motor control).

```
1      void motor_init(TIM_HandleTypeDef *htim) {
2          s_pwm_timer = htim;
3          HAL_TIM_PWM_Start(s_pwm_timer, TIM_CHANNEL_3); // Left
4              motor PWM
5          HAL_TIM_PWM_Start(s_pwm_timer, TIM_CHANNEL_4); // Right
6              motor PWM
7      }
```

Speed setting:

```
1      void motor_set_speed(motor_id_t motor, int16_t speed) {
2          // Clamp speed to [-MOTOR_PWM_MAX, MOTOR_PWM_MAX]
3          if (speed > MOTOR_PWM_MAX) speed = MOTOR_PWM_MAX;
4          else if (speed < -MOTOR_PWM_MAX) speed = -MOTOR_PWM_MAX;
5
6          uint16_t magnitude = (speed < 0) ? (uint16_t)(-speed) : (
7              uint16_t)speed;
8
9          // Set direction pins based on motor and sign of speed
10         if (motor == MOTOR_RIGHT) {
11             if (speed > 0) {
12                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_SET);
13                 // IN1
14                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_9, GPIO_PIN_RESET);
15                 // IN2
16             } else if (speed < 0) {
17                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_RESET);
18                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_9, GPIO_PIN_SET);
19             } else {
20                 // Speed = 0: stop (both low)
21                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_8, GPIO_PIN_RESET);
22                 HAL_GPIO_WritePin(GPIOD, GPIO_PIN_9, GPIO_PIN_RESET);
23             }
24         }
25         // Set PWM duty cycle
```

```

22     __HAL_TIM_SET_COMPARE(s_pwm_timer, TIM_CHANNEL_4,
        magnitude);
23 } else { // MOTOR_LEFT
24     // Similar using PD10,PD11 for direction and
        TIM_CHANNEL_3 for PWM
25     ...
26 }
27 }

```

How the motor driver works:

- A dual Hbridge motor driver needs two digital signals (IN1, IN2) for direction and a PWM signal for speed.
- When IN1=HIGH, IN2=LOW, the motor spins forward; opposite for backward; both LOW = brake/stop.
- The `__HAL_TIM_SET_COMPARE` macro updates the PWM compare register, setting the duty cycle. The timers ARR (autoreload) is 1599, so 01599 correspond to 0100% duty.

9.7 Encoder Driver (`encoder.c/h`)

9.7.1 Purpose

The robots motors have quadrature encoders that output pulse trains. We use input capture on two timer channels to measure the time between pulses, hence the rotation speed (RPM). This driver provides raw period and frequency.

9.7.2 Header

```

1  typedef struct {
2      uint32_t period_ticks;
3      float frequency_hz;
4      uint32_t pulse_count;
5  } encoder_data_t;
6
7  void encoder_init(TIM_HandleTypeDef *htim);
8  void encoder_handle_ic_callback(TIM_HandleTypeDef *htim);
9  encoder_data_t encoder_get_left_data(void);
10 encoder_data_t encoder_get_right_data(void);

```

9.7.3 Initialisation

```

1  void encoder_init(TIM_HandleTypeDef *htim) {
2      s_encoder_timer = htim;
3      HAL_TIM_IC_Start_IT(s_encoder_timer, TIM_CHANNEL_1); //
        Left encoder
4      HAL_TIM_IC_Start_IT(s_encoder_timer, TIM_CHANNEL_2); //
        Right encoder
5  }

```

Input capture interrupts are enabled, and the HAL will call `HAL_TIM_IC_CaptureCallback` on each rising edge.

9.7.4 Interrupt Callback

The `main.c` connects the actual interrupt to our handler:

```
1 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
2     encoder_handle_ic_callback(htim);
3 }
```

In `encoder_handle_ic_callback`, the code:

- Checks if the interrupt came from TIM2 (the timer used for encoders).
- Reads the captured timer value for the appropriate channel.
- Calculates the period (in ticks) since the last capture, handling timer overflow.
- Computes frequency = (timer clock) / period.
- Increments a pulse counter.

Frequency calculation requires knowing the timers input clock. The code dynamically determines the timer clock from RCC settings:

```
1 HAL_RCC_GetClockConfig(&clk_cfg, &flash_latency);
2 timer_clk_hz = HAL_RCC_GetPCLK1Freq();
3 if (clk_cfg.APB1CLKDivider != RCC_HCLK_DIV1) timer_clk_hz
4     *= 2U;
5 timer_clk_hz /= (htim->Init.Prescaler + 1U);
```

This ensures correct frequency even if the clock tree changes.

9.7.5 Reading Data

The getter functions (`encoder_get_left_data`) disable interrupts temporarily to read the struct safely, preventing a partially updated value.

9.8 Main Application (`main.c`)

This is where everything comes together. We'll go through the significant parts step by step.

9.8.1 Configuration and Global Variables

```
1 #define CONTROL_LOOP_HZ 200.0f
2 #define CONTROL_DT (1.0f / CONTROL_LOOP_HZ) // 0.005 s
3 #define TELEMETRY_DIVIDER 10U // send
4     telemetry every 10 control cycles
5
6 volatile float g_angle_deg, g_gyro_dps;
7 volatile int16_t g_motor_cmd;
```

```

7     volatile uint8_t telemetry_flag = 1;
8     volatile uint8_t g_uart_busy = 0U;

```

‘volatile’ tells the compiler these variables can be changed in an interrupt, so it must reload them every time.

9.8.2 System Initialisation: start_robot_modules()

```

1     static void start_robot_modules(void) {
2         pid_gains_t gains = { .kp = 55.0f, .ki = 1.2f, .kd = 2.8f
3             ,
4             .integral_limit = 60.0f, .output_limit = MOTOR_PWM_MAX
5         };
6         mpu6050_init(&hi2c2);
7         mpu6050_calibrate(&hi2c2, 300U);
8         angle_init(&hi2c2);
9         pid_init(gains);
10        encoder_init(&htim2);
11        motor_init(&htim2);
12        HAL_TIM_Base_Start_IT(&htim6);    // start the control
13        loop timer
14    }

```

The order matters: sensor must be initd and calibrated before the angle filter and PID are used.

9.9 The Control Loop (TIM6 Interrupt)

TIM6 is configured to fire at 200 Hz. Inside HAL_TIM_PeriodElapsedCallback:

```

1     void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2     {
3         if (htim->Instance != TIM6) return;
4
5         if (angle_update(CONTROL_DT) != HAL_OK) return;
6         g_angle_deg = angle_get_deg();
7         g_gyro_dps = angle_get_gyro_rate_dps();
8         float command = pid_compute(0.0f, g_angle_deg, CONTROL_DT
9             );
10        g_motor_cmd = (int16_t)command;
11        motor_set_speed(MOTOR_LEFT, g_motor_cmd);
12        motor_set_speed(MOTOR_RIGHT, g_motor_cmd);
13
14        g_control_ticks++;
15        if ((g_control_ticks % TELEMETRY_DIVIDER) == 0U)
16            telemetry_flag = 1U;
17    }

```

Heres the sequence:

1. Update the angle using the fixed time step CONTROL_DT.
2. Retrieve the angle and gyro rate (for telemetry).

3. Run the PID controller with `setpoint = 0` (upright).
4. Cast the float command to an `int16_t` and send it to both motors.
5. Every 10th call, set `telemetry_flag` to request a UART transmission.

9.9.1 Telemetry

The main loop (in `while(1)`) checks `telemetry_flag` and calls `send_telemetry_nonblocking()`. That function uses `myPrintf` to send a CSV line: angle, gyro rate, motor command, left encoder frequency, right encoder frequency. It avoids blocking by using a busy flag and the UART transmit complete callback.

9.9.2 Additional Interrupt Service Routines

The encoder callback is linked:

```
1 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
2     encoder_handle_ic_callback(htim);
3 }
```

UART transmit complete callback resets the busy flag:

```
1 void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
2     if (huart->Instance == USART2) g_uart_busy = 0U;
3 }
```

9.10 Summary

Youve now seen the entire firmware of a selfbalancing robot. Each module has a clear responsibility:

- `mpu6050` lowlevel sensor communication.
- `complementary` angle estimation (sensor fusion).
- `pid` control algorithm.
- `motor` actuator interface.
- `encoder` speed measurement.
- `main` orchestrates everything with precise timing.

This modular structure makes the code reusable, testable, and easier to understand. Feel free to experiment by changing the PID gains or filter coefficients and observing the effect on the robots behavior.